

Repository Plan for `mini-llm-lab`

Project design document

September 26, 2026

1 Purpose of the repository

This repository is meant to be a learning platform for building a small modern **large language model (LLM)** from the ground up while keeping the mathematics, the matrices, and the numerical flow visible.

The central idea is:

Every important part of the model should be inspectable as explicit matrices, explicit tensor operations, and explicit backward derivatives.

The project is therefore *not* primarily about obtaining the strongest model possible. Instead, it is about understanding:

- how text becomes tokens,
- how tokens become vectors,
- how **Transformer** blocks propagate information,
- how **attention** works,
- how residual connections work,
- how the next-token prediction loss is formed,
- how gradients move backward through the model, and
- how optimization updates the matrices.

2 Main design philosophy

The repository follows the following rules.

Rule 1: explicit numerical objects

All trainable weights should appear as visible matrices, usually through a small `Parameter` class with:

`data,` `grad.`

Rule 2: explicit forward pass

Each module should compute its forward operation with explicit tensor contractions, for example:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V,$$

rather than hiding the operation behind a high-level neural-network framework layer.

Rule 3: explicit backward pass

Each module should also contain its own hand-derived backward pass. For example, if

$$Y = XW,$$

then the backward formulas are:

$$\frac{\partial L}{\partial W} = X^\top \frac{\partial L}{\partial Y}, \quad \frac{\partial L}{\partial X} = \frac{\partial L}{\partial Y} W^\top.$$

Rule 4: CPU first, GPU second

The same code should run with:

- **NumPy** for a clear **central processing unit (CPU)** reference implementation,
- **CuPy** for **graphics processing unit (GPU)** acceleration.

Rule 5: verify before scaling

Every operator should be tested numerically before being integrated into a larger model. In practice this means:

- directional derivative checks,
- finite-difference tests,
- shape checks,
- causality checks for attention,
- and small overfitting experiments.

3 Acronyms and terms

Term	Meaning
LLM	Large language model. A neural network trained to predict the next token in text.
CPU	Central processing unit. The ordinary processor of the computer.
GPU	Graphics processing unit. A highly parallel processor used for accelerated tensor operations.
BPE	Byte-pair encoding. A tokenizer method that merges frequent token pairs to build a vocabulary.
LM	Language model. In this repository, usually a decoder-only next-token prediction model.
GQA	Grouped-query attention. A variant of attention where several query heads share the same key and value heads.
RoPE	Rotary positional embedding, more precisely a rotary positional transformation applied to query and key vectors.
RMSNorm	Root-mean-square normalization. A normalization layer using the root mean square of the input vector.

FFN	Feed-forward network. In this project, the position-wise non-attention block inside a Transformer block.
SwiGLU	Swish Gated Linear Unit. A feed-forward structure using a SiLU (sigmoid linear unit) gate.
MoE	Mixture of experts. A sparse feed-forward structure in which a router activates only some experts for each token.
KV cache	Key-value cache. Saved keys and values used during autoregressive generation to avoid recomputing the whole prefix.
CE loss	Cross-entropy loss. The loss used for next-token prediction.
AdamW	An optimization method based on Adam with decoupled weight decay.
SFT	Supervised fine-tuning. A later-stage training step used to teach instruction-following behavior.

4 Repository structure

The repository is planned to evolve toward the following structure:

```

mini_llm_lab/
|-- mini_llm/
|   |-- backend.py
|   |-- config.py
|   |-- init.py
|   |-- parameter.py
|   |-- ops/
|   |   |-- linear.py
|   |   |-- embedding.py
|   |   |-- rmsnorm.py
|   |   |-- rope.py
|   |   |-- attention.py
|   |   |-- swiglu.py
|   |   |-- loss.py
|   |-- optim/
|   |   |-- adamw.py
|   |   |-- grad_clip.py
|   |   |-- schedule.py
|   |-- diagnostics/
|   |   |-- gradcheck.py
|   |-- blocks/
|   |   |-- transformer_block.py
|   |   |-- moe.py
|   |-- model/
|   |   |-- decoder_lm.py
|   |   |-- inference.py
|   |-- data/
|   |   |-- tokenizer.py
|   |   |-- preprocess.py
|   |   |-- dataloader.py
|-- tests/
|-- examples/

```

`'-- docs/`

5 Planned milestones

Milestone 1: numerical primitives (already started)

Implemented or being implemented:

- backend selection (NumPy / CuPy),
- parameter objects,
- initialization,
- linear layers,
- embedding lookup,
- RMSNorm,
- RoPE,
- grouped-query causal attention,
- SwiGLU,
- cross-entropy loss,
- gradient clipping,
- AdamW,
- numerical derivative checks.

Milestone 2: dense Transformer block

Build the standard dense Transformer block:

$$X \rightarrow \text{RMSNorm} \rightarrow \text{Attention} \rightarrow +X \rightarrow \text{RMSNorm} \rightarrow \text{SwiGLU} \rightarrow +X.$$

Milestone 3: dense decoder language model

Stack multiple Transformer blocks, tie the input embedding matrix to the output language-model head, and train a small decoder-only language model on a tiny corpus.

Milestone 4: sparse feed-forward replacement

Replace the dense feed-forward network by a **mixture of experts (MoE)** block:

$$\text{Dense FFN} \longrightarrow \text{Router} + \text{Experts}.$$

Milestone 5: data pipeline

Build the tokenizer and dataset pipeline:

- raw documents,
- training/validation split,
- tokenizer training,
- token packing,
- binary token shards.

Milestone 6: full training loop

Create:

- checkpoints,
- logging,
- learning-rate schedules,
- validation,
- and loss/gradient statistics.

Milestone 7: inference

Add:

- autoregressive decoding,
- sampling,
- key-value cache,
- prompt processing.

Milestone 8: research experiments

Compare:

- dense vs. MoE,
- AdamW vs. Muon,
- data mixtures,
- and routing behavior.

6 Model overview

The full model will be a decoder-only Transformer for next-token prediction.

6.1 Input text and tokenization

A raw text sequence such as

"The fluid velocity increases near the wall."

is converted to discrete token identifiers using a tokenizer such as **byte-pair encoding (BPE)**:

$$[t_0, t_1, \dots, t_{T-1}].$$

6.2 Embedding

Each token identifier is converted into a vector through an embedding matrix

$$E \in \mathbb{R}^{V \times d_{\text{model}}},$$

where:

- V is the vocabulary size,
- d_{model} is the model dimension.

If X_0 denotes the embedded token sequence, then

$$X_0 \in \mathbb{R}^{B \times T \times d_{\text{model}}},$$

where B is the batch size and T is the sequence length.

6.3 Transformer block

Each Transformer block contains two residual branches.

Attention branch.

$$\begin{aligned} Z_1 &= \text{RMSNorm}(X), \\ Q &= Z_1 W_Q, \quad K = Z_1 W_K, \quad V = Z_1 W_V. \end{aligned}$$

Then RoPE is applied to Q and K . Next, causal grouped-query attention is evaluated:

$$S = \frac{QK^\top}{\sqrt{d_h}} + M,$$

where M is the causal mask. The attention probabilities are

$$P = \text{softmax}(S),$$

and the context is

$$C = PV.$$

After output projection,

$$A = CW_O,$$

the residual update gives:

$$X' = X + A.$$

Feed-forward branch.

$$Z_2 = \text{RMSNorm}(X'),$$

then the SwiGLU feed-forward network computes:

$$\begin{aligned} G &= Z_2 W_{\text{gate}}, & U &= Z_2 W_{\text{up}}, \\ H &= \text{SiLU}(G) \odot U, \\ F &= H W_{\text{down}}. \end{aligned}$$

The second residual update gives:

$$Y = X' + F.$$

6.4 Stacking

If there are L layers, the sequence passes through:

$$X_0 \rightarrow X_1 \rightarrow X_2 \rightarrow \cdots \rightarrow X_L.$$

6.5 Final normalization and logits

After the last block:

$$Z_{\text{final}} = \text{RMSNorm}(X_L).$$

The logits are then computed using the tied embedding matrix:

$$\text{logits} = Z_{\text{final}} E^\top.$$

6.6 Next-token prediction

For each position t , the model predicts the next token:

$$p(x_{t+1} \mid x_{\leq t}) = \text{softmax}(\text{logits}_t).$$

Training uses the mean cross-entropy loss over all positions.

7 Numerical conventions and initialization

The initial implementation uses:

- clipped normal initialization for most matrices,
- smaller standard deviation for residual-output matrices,
- $\gamma = 1$ for RMSNorm gains,
- AdamW optimization,
- global gradient clipping,
- warmup and cosine learning-rate decay.

The main reason is not fashion; it is numerical stability and interpretability. A transparent project is easier to debug when:

- activation scales are monitored,
- gradient scales are monitored,
- the optimizer is simple and explicit,
- and each matrix has a clear role.

8 How training will be validated

Before any serious training run, the repository should verify:

1. **Operator checks:** each layer passes local finite-difference checks.
2. **Block checks:** a full Transformer block passes directional derivative checks.
3. **Model checks:** the full dense model passes end-to-end gradient tests on a tiny example.
4. **Memorization checks:** the model can intentionally overfit a tiny corpus.
5. **Scaling checks:** only after the above are correct do we train on larger corpora.

9 Planned future MoE extension

Later, the dense feed-forward branch will be replaced by a sparse MoE block.

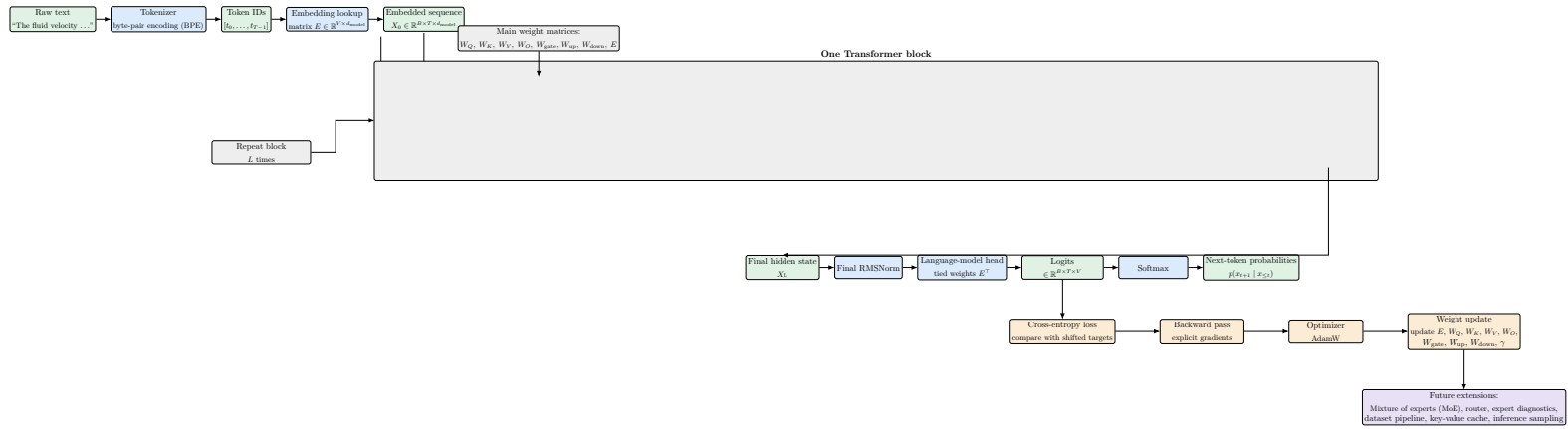
The idea is:

- a router computes a score for each expert,
- only the top- k experts are selected,
- the selected expert outputs are combined,
- and this replaces the ordinary dense feed-forward transformation.

This makes it possible to study:

- sparse conditional computation,
- expert specialization,
- routing statistics,
- load balancing,
- and domain-sensitive expert behavior.

Full model diagram



10 Conclusion

This repository is intended to become a transparent experimental platform for understanding the full life of a language model:

raw text $\rightarrow$ tokens $\rightarrow$ embeddings $\rightarrow$ Transformer blocks $\rightarrow$ logits $\rightarrow$ cross-entropy loss $\rightarrow$ gradients $\rightarrow$ weight updates.

The project starts with dense explicit components, validates them carefully, and only then moves toward modern sparse mechanisms such as MoE routing and expert specialization.